

Frontend, Electron & Onboarding — Explained

A plain-English guide to the parts of FluencyOS built on React, Electron, Vite, Tailwind CSS and TypeScript — written assuming no prior exposure to any of these tools, only general programming and HTML/CSS knowledge.

Who this is for. You already know how to program, and you know HTML and CSS. You have not necessarily used React, Electron, Vite, Tailwind CSS, TypeScript, or Zustand before. Every section below introduces a new tool by comparing it to something you already know, before showing the real code from this project that uses it. By the end, you should be able to explain to your teacher *what* each piece is, *why* it exists, and *how* a user's click travels from the screen down to a saved database row during onboarding.

Contents

1. What kind of app is this, physically? — desktop app, not a website
2. The five tools, one at a time — Node.js, TypeScript, React, Vite, Tailwind CSS
3. Electron: two programs pretending to be one — main process vs renderer
4. The security bridge — preload, contextBridge, the handshake token
5. Touring the frontend folder — features, stores, App.tsx
6. The onboarding wizard, step by step — the centerpiece of this guide
7. What's really wired up vs. what's a mock — an honest map
8. Running it live, and a glossary — for the demo and Q&A

1. What kind of app is this, physically?

FluencyOS looks like a normal desktop program — it has its own window, its own icon, a title bar with minimize/maximize/close buttons, and it keeps running without a browser open. But under the hood, the entire visible interface is a webpage: HTML, CSS and JavaScript, exactly like a website. The trick is a tool called **Electron**, which packages a stripped-down Chrome browser and a copy of Node.js together into one program, so that a webpage can be shown as if it were a native app, and — unlike a normal webpage — that page is allowed to open native file-picker dialogs, spawn other programs on your computer, and read your hardware specs.

So there are really **two halves** to this project, and your teacher will care about both:

HALF 1 – THE BACKEND

A Python (FastAPI) server, started automatically in the background, that owns the SQLite database and does the real work (scoring the placement test, saving the user's profile, checking hardware capability). Not the focus of this document.

HALF 2 – THIS DOCUMENT

The **Electron shell** (the desktop-app wrapper) and the **renderer** (the React webpage running inside that window) — including the five-step onboarding wizard a new user completes the first time they open the app.

2. The five tools, one at a time

None of these tools change what a browser fundamentally does with HTML/CSS/JS. Each one solves an annoyance you'd eventually hit while hand-writing a large app the plain way. Read each comparison, then look at the "in this project" line to see it isn't abstract.

2.1 Node.js — JavaScript outside the browser

You already know JavaScript as "the language that runs inside a webpage." Node.js is the same language, running as a normal program on your computer instead — it can read files, start other programs, and open network ports, the way Python or a compiled program can. Electron's desktop-app half (see Section 3) is a Node.js program. So are the build scripts that assemble this project.

2.2 TypeScript — JavaScript with a spell-checker for types

TypeScript is JavaScript with one thing bolted on: before your code ever runs, a checker reads it and complains if, say, a function expecting a number is handed a piece of text. It compiles down to ordinary JavaScript — the browser or Node.js never sees TypeScript itself, only its plain-JS output. Every `.ts` / `.tsx` file in this project is really "JavaScript plus type annotations."

```
// Plain JavaScript – this typo is only discovered when the app crashes at runtime
function assessHardware(cpuCores, totalRamBytes) { /* ... */ }
assessHardware("4", 6_000_000_000); // "4" the string slips through silently

// TypeScript – the same typo is a red squiggly line before you even save the file
function assessHardware(cpuCores: number, totalRamBytes: number) { /* ... */ }
assessHardware("4", 6_000_000_000); // x Argument of type 'string' is not assignable to
'number'
```

2.3 React — describing the screen instead of editing it by hand

In plain HTML/JS, you update the page by finding an element and mutating it:

`document.getElementById('score').textContent = newScore;` That's fine for a small page, but it gets unmanageable once dozens of things on screen can each change independently. React flips the approach: you write a function that says *"given this data, here is what the whole*

screen should look like," and React figures out which bits of real HTML actually need to change, and updates only those.

THE WAY YOU ALREADY KNOW

```
const el =
document.getElementById('name');
el.textContent = user.name;
el.classList.toggle('empty',
!user.name);
```

THE REACT WAY

```
<div className={user.name ? '' :
'empty'}>
  {user.name}
</div>
```

That HTML-looking snippet inside JavaScript is called **JSX** — it is not a new language, it is JavaScript that lets you write tags directly, which then get converted into plain `document.createElement`-style calls behind the scenes. Curly braces `{ }` drop back into normal JavaScript expressions, exactly like a template literal's ``${ }``.

A **component** is just a JavaScript function that returns some JSX — think of it as a custom HTML tag you invent yourself. This project has one component per screen (e.g. `Dashboard()`, `Review()`) and smaller reusable ones inside them (e.g. a `TextField` component used for every text input in the onboarding form, shown in Section 6).

REACT WORD	WHAT IT MEANS, IN YOUR EXISTING VOCABULARY
component	A function that returns some UI — like a custom, reusable HTML tag you define yourself.
props	The arguments passed into that function — like attributes on an HTML tag (<code><TextField label="Name" /></code>).
state	A variable that, when changed, tells React "re-run this function and update the screen." Plain JS variables don't do that on their own.
hook	A special function (always starting with <code>use...</code> , e.g. <code>useState</code> , <code>useEffect</code>) that plugs a component into React's re-rendering machinery.

2.4 Vite — a smarter local web server

You've probably run something like `python -m http.server` to preview a static site locally. Vite is that idea, upgraded: it serves the app on `http://127.0.0.1:5173`, but it also **instantly refreshes the browser the moment you save a file** ("hot reload" — no manual refresh), and it understands TypeScript/JSX and converts them on the fly. When you're ready to ship, `vite build` switches modes and instead bundles every file into a handful of optimized, minified `.js` / `.css` files under `renderer/dist/` — the same idea as concatenating and minifying scripts by hand, automated.

2.5 Tailwind CSS — writing style rules as class names

Normally you write a CSS rule once and reference it by a class name you invent:

PLAIN CSS

```
.toolbar {  
  display: flex;  
  gap: 8px;  
  padding: 12px;  
}
```

...then in HTML: `<div class="toolbar">`

TAILWIND CSS

```
<div className="flex gap-2 p-3">
```

No separate CSS rule to name or maintain — each utility class already means one property. `flex` = `display:flex`, `gap-2` = `gap:8px`, `p-3` = `padding:12px`.

Tailwind still compiles down to a real `.css` file — nothing magic happens at runtime — it just means you rarely hand-write new CSS rules; you compose existing utility classes instead, which keeps every component's styling visible right next to its markup instead of split into a separate stylesheet.

This project's twist: the actual pixel/color values aren't hardcoded into Tailwind's config — they're CSS custom properties (`--acc`, `--bg`, `--tx`, ...) defined once in `renderer/src/styles/globals.css`, and Tailwind is configured to treat them as color names (`text-tx`, `bg-acc`). Toggling dark/light theme just swaps which set of variable values is active — every component using `text-tx` updates automatically, without any component needing to know which theme is active.

2.6 Zustand — one shared object every component can read and write

In plain JS, if two unrelated parts of a page need to share a value (say, "which theme is active"), you'd reach for a global variable, or pass it down manually through every function in between. React discourages plain global variables because it wouldn't know when to re-render.

Zustand is a tiny library that gives you a global-variable-like "store" that *does* notify React when it changes.

```
// Anywhere in the app, any component can do this – no manual passing-down required:  
const theme = useShellStore(s => s.theme); // read one field  
const toggleTheme = useShellStore(s => s.toggleTheme); // read one action
```

This project has three such stores: `appStore` (is a user logged in, is onboarding done), `shellStore` (which screen is showing, dark/light theme, is the sidebar collapsed), and `onboardingStore` (every field of the five-step wizard, plus the functions that validate and save each step — the star of Section 6).

3. Electron: two programs pretending to be one

The single most important idea to explain to your teacher is that **an Electron app is really two separate JavaScript programs running at the same time**, with a strict wall between them:

MAIN PROCESS

A plain **Node.js** program. Full access to the operating system: it can read/write files, spawn other programs, open native dialogs. There is exactly one of these per app. In this project it lives under `electron/main/`.

RENDERER PROCESS

A **Chromium browser tab** showing the React app. By default it can do everything a normal webpage can — and, deliberately, *nothing more*. It cannot touch the file system or spawn processes directly. Lives under `renderer/src/`.

Why the wall? Security. A normal website you visit in Chrome cannot read your files or run programs on your machine — that sandbox is exactly what keeps a malicious webpage from wrecking your computer. Electron keeps that same sandbox around the renderer on purpose, even though the app is "trusted" here, because a bug or a future dependency's bug in the React code should still not be able to reach the file system directly.

3.1 What happens when you double-click the app

- 1 **Electron's main process starts** and waits until it's ready (`app.whenReady()` in `electron/main/index.ts`).
- 2 **It launches the Python backend** as a child process (`startBackend()`) — picks a free port, generates a random secret token (Section 4.2), and waits (polling `/health`) until FastAPI answers.
- 3 **It creates the window** (`createWindow()`) — a frameless `BrowserWindow`, sized 1280×860, with a custom preload script attached.
- 4 **It loads the page.** While developing, that's literally `http://127.0.0.1:5173` — the Vite dev server from Section 2.4, so you get hot reload while working. In a built/shipped app it instead loads the pre-built `renderer/dist/index.html` file straight off disk — no server involved at all.
- 5 **On quit** (`before-quit`), the main process kills the backend child process it started, so no orphaned Python server keeps running after the window closes.

```
// electron/main/index.ts – the entire boot sequence, trimmed
app.whenReady().then(async () => {
  backendHandle = await startBackend(); // step 2
  await createWindow();                // steps 3-4
});

app.on('before-quit', () => {
  stopBackend(backendHandle);          // step 5
});
```

Notice this file barely does anything itself — it delegates to `backend-process.ts` (spawning/health-checking the Python server) and `ipc-handlers.ts` (Section 4). That split — one small file per responsibility — is the same discipline you'd apply to any codebase, just applied to Electron's main process.

4. The security bridge: how the renderer is allowed to ask for anything at all

If the renderer (the React webpage) has *no* direct access to the file system or the OS, how does the onboarding wizard show a native "choose a folder" dialog, or read how much RAM your computer has? Through a narrow, explicitly-listed gate called the **preload script**.

4.1 The preload script and `contextBridge`

Think of it like a hotel mini-bar: you, the guest (the renderer), cannot walk into the hotel's main storage room and take whatever you want. But the hotel places a small, curated set of items in your room that you *are* allowed to use. The preload script is that mini-bar stocking list — it runs with slightly more privilege than the renderer, and it deliberately exposes only a handful of specific functions onto `window.fluencyos`, nothing more:

```
// electron/preload/index.ts – the entire allowed API surface, verbatim
contextBridge.exposeInMainWorld('fluencyos', {
  getBackendInfo: () => ipcRenderer.invoke('backend:get-info'),
  getSystemInfo: () => ipcRenderer.invoke('system:get-info'),
  pickDataFolder: () => ipcRenderer.invoke('dialog:pick-folder'),
  pickBookFiles: () => ipcRenderer.invoke('dialog:pick-book-files'),
  getPathForFile: (file) => webUtils.getPathForFile(file),
  minimizeWindow: () => ipcRenderer.send('window:minimize'),
  maximizeWindow: () => ipcRenderer.send('window:maximize'),
  closeWindow: () => ipcRenderer.send('window:close'),
});
```

That's the entire list. The React code cannot ask for anything not on this list — there is no `window.fs.readFile` or similar lying around. Every call above is really a message sent to the main process (`ipcRenderer.invoke` / `.send` — Electron's Inter-Process Communication), which the main process answers in `electron/main/ipc-handlers.ts`:

```
// electron/main/ipc-handlers.ts – abbreviated
ipcMain.handle('system:get-info', () => getSystemInfo()); // real
os.cpus()/os.totalmem()
ipcMain.handle('dialog:pick-folder', async () => {
  const result = await dialog.showOpenDialog(win, { properties: ['openDirectory',
    'createDirectory'] });
  return result.canceled ? null : result.filePaths[0];
});
ipcMain.on('window:minimize', () => getWindow()?.minimize());
```

This is the concrete answer to "how does a webpage open a native folder-picker dialog?" — it doesn't; it politely asks the main process to do it, and receives the chosen path back as a plain string, through this narrow bridge.

4.2 The handshake token — why the local server has a password

The Python backend listens on a real HTTP port on your machine (e.g. `127.0.0.1:34233`). Any other program running on the same computer could, in principle, guess that port and start asking it for your data. To close that gap, the main process generates a random 32-byte secret **once per launch** (never written to disk, never logged) and requires every request to the backend to include it:

```
// electron/main/handshake.ts
export function generateHandshakeToken(): string {
  return randomBytes(32).toString('hex');
}
```

```
// renderer/src/lib/apiClient.ts — every single API call goes through this one function,
// which attaches the token automatically so no screen component has to remember to
async function request(path, init) {
  const { baseUrl, token } = requireBackendInfo();
  const res = await fetch(`${baseUrl}${path}`, {
    ...init,
    headers: { 'Content-Type': 'application/json', 'X-FluencyOS-Token': token, ...
(init?.headers ?? {}) },
  });
  /* ... */
}
```

The renderer only ever learns this secret by asking the main process for it (`window.fluencyos.getBackendInfo()` → `backend:get-info` handler), which is itself gated by the preload bridge above. A webpage running in some other browser tab has no way to obtain it.

5. Touring the frontend folder

`renderer/src/` is a normal React + TypeScript project — nothing Electron-specific about its structure. It's organized "by feature," not "by file type":

```
renderer/src/
App.tsx           ← the traffic controller (Section 5.3)
store/           ← Zustand stores (Section 2.6)
  appStore.ts     is a user signed in? is onboarding done?
  shellStore.ts   which screen is showing, theme, sidebar state
  onboardingStore.ts the 5-step wizard's data + save logic (Section 6)
lib/
  apiClient.ts    the one place that talks to the Python backend
features/        ← one folder per screen
  onboarding/     the wizard (Section 6)
  dashboard/, player/, reader/, vocabulary/, review/,
  conversation/, challenge/, forest/, settings/, shell/
    each usually has: SomeScreen.tsx (the UI, a React component)
                      someMockData.ts (sample data — see Section 7)
```

5.1 Why "mock data" files exist at all

Several screens (Conversation, Forest, the Scene Challenge, Review) depend on features that don't have a real backend yet — there is no AI conversation engine, no spaced-repetition scheduler, no embeddings model wired up. Rather than leave those screens blank, each has a small `*MockData.ts` file holding realistic-looking sample data, with a comment at the top saying so honestly. This lets the whole UI be demoed end-to-end today, while making it obvious in the code (and to a reviewer) exactly which parts are real. Section 7 lists which is which.

5.2 A real component, read top to bottom

Here is one of the smaller, self-contained components in the project — a labeled text input used on the first onboarding step — annotated line by line, since it uses almost every React idea from Section 2.3 at once:

```
function TextField({ label, sub, value, onChange, placeholder }) { // a component = a
function...
  return ( // ...that returns JSX
    <div className="flex items-center justify-between ..." > // Tailwind utility
      classes
        <div>
          <div>{label}</div> // {label} = a prop, in
        braces
          <div>{sub}</div>
        </div>
        <input
          value={value} // controlled input:
          React owns the value
          onChange={(e) => onChange(e.target.value)} // same DOM event you
          already know
        />
      </div>
    );
  }
```

It's used like a custom HTML tag, passing data in as attributes (props):

```
<TextField
  label="Display name"
  value={profile.displayName} // current value, read from the Zustand store
  onChange={(v) => updateProfile({ displayName: v })} // writes back to the store on every
  keystroke
/>
```

5.3 App.tsx — the traffic controller

There is no URL-based router in this app (no `/dashboard`, `/settings` in an address bar — it's a desktop window, not a browser). Instead, `shellStore` just holds a plain string field called `screen` (e.g. `'dashboard'`, `'review'`), and `App.tsx` reads it and picks which component to show with an ordinary `switch` statement:

```
function ScreenContent() {
  const screen = useShellStore((s) => s.screen);
  switch (screen) {
    case 'dashboard': return <Dashboard />;
    case 'review':     return <Review />;
    // ...one case per screen...
    default: return <ComingSoon />;
  }
}
```

Clicking a sidebar link just calls `goScreen('review')`, which sets that one string in the store — Zustand then tells React "something changed," React re-runs `ScreenContent`, and the switch statement now returns a different component. No page reload, no network round-trip, because it was never really "navigation" in the browser sense — just one variable changing and React reacting to it.

One level up, `App()` itself makes an even bigger either/or decision, and this is the exact switch that ends onboarding and enters the real app:

```
{!initError && backendReady && onboardingCompleted === false && <OnboardingWizard />}
{!initError && backendReady && onboardingCompleted === true  && <MainApp />}
```

6. The onboarding wizard, step by step

This is the flow you'll most likely want to walk your teacher through live. It is five steps, each its own component, all orchestrated by one shell component and one Zustand store.

STEP 1 · PROFILE

Name, native language, target language, data folder.

STEP 2 · PLACEMENT

A short CEFR-level quiz to estimate the learner's current level.

STEP 3 · ENGINE

Checks whether this computer can run a local AI model well, or should use an API key instead.

STEP 4 · HABIT

Daily goals (reviews, conversation minutes, reading, etc.) and quiet hours.

STEP 5 · COMPANION

Pick a companion character; finishing this step marks onboarding complete.

6.1 The shell: `OnboardingWizard.tsx`

This component doesn't know anything about what's *inside* each step — it only knows there are five of them. It keeps a lookup table from step number to component, and renders whichever one is current:

```
const STEP_COMPONENTS = { 1: StepProfile, 2: StepPlacement, 3: StepEngine, 4: StepHabit, 5: StepCompanion };

export function OnboardingWizard() {
  const step = useOnboardingStore((s) => s.step);           // current step number, from the
store
  const StepComponent = STEP_COMPONENTS[step];             // look up which component to
show
  // ...renders the progress rail, current step's title/body text, then:
  return (<> <StepComponent /> </>);                       // <StepComponent /> – a variable
used as a tag!
}
```

That last line is worth pointing out to your teacher: in JSX, a component stored in a normal JavaScript variable can be rendered by writing it as a tag, exactly like the hardcoded `<div>` tags elsewhere. This is what lets one small shell file support five completely different steps without an if/else chain for each one.

6.2 The store: one object holding the whole wizard's state

`onboardingStore.ts` holds every field the five steps need (`profile`, `placement`, `hardware`, `engine`, `habit`, `companion`), plus a `submission` field tracking whether a save is in flight or failed. Every step reads straight from this one store and calls one of its update functions — nothing is passed down through props from parent to child the "plain React" way, which is exactly the problem Zustand (Section 2.6) exists to avoid.

6.3 Walking through Step 1 concretely — from click to database row

Here is the entire path a single keystroke takes, in order:

- 1 User types into the "Display name" input. `StepProfile.tsx`'s `TextField` fires its `onChange`.
- 2 That calls `updateProfile({ displayName: v })`, which does an **immutable update** — it builds a brand-new profile object by spreading the old one and overwriting one field, rather than mutating the old object in place:

```
updateProfile: (patch) => set((s) => ({ profile: { ...s.profile, ...patch } })))
```

- 3 Zustand notifies every component reading `profile` that it changed. React re-renders the input with the new value — this is what makes it a "controlled input": the on-screen text is always exactly what's in the store, never out of sync.
- 4 User clicks **Continue**. This calls `goNext()`, which first runs `validateStep(state)` — for step 1, that's just "is the trimmed display name empty?" If so, it sets `submission.error` and stops; the wizard shell displays that message and nothing is sent anywhere yet.
- 5 If valid, it builds a plain object matching what the backend expects and calls `api.post('/users', payload)` — which, remember from Section 4.2, automatically carries the secret handshake token. FastAPI creates the row in SQLite and returns the saved user (now with a real database ID).

- 6 The store saves that ID (`set({ userId: user.id })`) — every later step needs it to know *which* user row to update — and advances `step` from 1 to 2. The wizard shell's lookup table swaps `StepProfile` out for `StepPlacement`.

```
// onboardingStore.ts – goNext(), the real code for step 1, trimmed to the essentials
goNext: async () => {
  const validationError = validateStep(get());
  if (validationError) { set({ submission: { status: 'error', error: validationError } });
  return; }

  set({ submission: { status: 'submitting', error: null } });
  try {
    if (state.step === 1) {
      const user = await api.post('/users', {
        display_name: state.profile.displayName.trim(),
        native_language: state.profile.nativeLanguage.trim(),
        target_language: state.profile.targetLanguage.trim(),
        data_folder: state.profile.dataFolder.trim(),
      });
      set({ userId: user.id });
    }
    // ...similar blocks for steps 2, 4, 5 – see 6.4...
    set((s) => ({ step: s.step + 1, submission: { status: 'idle', error: null } }));
  } catch (err) {
    set({ submission: { status: 'error', error: err.message } }); // network/server errors
    surface the same way
  }
}
```

6.4 What each step actually saves

STEP	WHAT HAPPENS ON "CONTINUE"	BACKEND CALL
1 · Profile	Creates the user row for the first time.	POST /users
2 · Placement	Saves the CEFR level — either self-reported, or the estimate computed from the placement quiz the user just took.	PATCH /users/:id/placement
3 · Engine	No save on "Continue" — instead, as soon as this step loads, it silently asks the main process for real CPU/RAM numbers and posts them to the backend to get a capability verdict (Section 6.5). Validation only blocks if local mode is chosen with no tier selected.	POST /engine/assess-hardware
4 · Habit	Saves Step 3's engine choice <i>and</i> Step 4's daily-goal settings together in one call.	PUT /users/:id/settings
5 · Companion	Saves the chosen companion, then immediately marks onboarding complete — which is the exact moment <code>App.tsx</code> switches from the wizard to the main app (Section 5.3).	POST /users/:id/companion POST /users/:id/onboarding/complete

6.5 Step 3 is the one genuinely "smart" step

Every other step just collects and saves form input. Step 3 additionally makes a real decision using real data from the machine it's running on:

- 1 On mount, it calls `loadHardwareInfo()`, which asks the main process (through the preload bridge, Section 4.1) for the real CPU core count and total RAM — `os.cpus().length` and `os.totalmem()`, actual Node.js OS calls, not sample data.
- 2 It sends those two numbers to the Python backend's `/engine/assess-hardware` endpoint, which runs a small pure function deciding whether a "Light," "Balanced," or "Heavy" local AI model tier is realistic for this hardware, or whether no local tier is realistic at all.
- 3 If nothing local is realistic, the store automatically pre-selects "use an API key instead" mode — the user can still override it manually, but they aren't steered toward a model their computer can't actually run smoothly.

Why this is worth highlighting to a teacher: it's the one onboarding step that isn't just "a form that saves what you typed" — it's the frontend, Electron, and backend all cooperating on one real decision, using a number that couldn't have been faked ahead of time (this machine's actual RAM).

7. What's really wired up vs. what's a mock — an honest map

Being upfront about this distinction is a good habit in any project, and it's an easy, confident answer if a teacher asks "does this actually work end-to-end?":

AREA	STATUS
Onboarding (all 5 steps)	Real. Every step other than the wizard shell's own UI state reads/writes the actual SQLite database through FastAPI, including the placement quiz scoring and the hardware capability check.
Electron shell (window, title bar, backend process management, IPC bridge)	Real. This is genuine desktop-app plumbing — the security bridge, the handshake token, and the backend health-check are all live code, not simulated.
Dashboard, Vocabulary, Learn by Watching/Reading, Review, Settings	Real for what has a backend (book import/reading, vocabulary storage), mock for the rest (e.g. FSRS scheduling in Review is simulated locally, since no spaced-repetition engine exists yet).
Conversation, Session Report, Scene Challenge, Forest	Visual mock only. There is no LLM/speech pipeline or embeddings model wired up yet, so these screens are built against realistic sample data, clearly marked as such in each screen's <code>*MockData.ts</code> file, so the UI can be demoed and reviewed today without pretending a backend exists that doesn't yet.

8. Running it live, and a glossary for Q&A

8.1 The one command

```
npm run dev
```

This single script (`scripts/dev.mjs`) does three things in order: starts the Vite dev server (Section 2.4), waits until it responds, then launches Electron pointed at it — and Electron's own startup (Section 3.1) spawns the Python backend itself. While it's running: **changes to renderer files hot-reload instantly** (no restart needed); changes to Electron's `main/` or `preload/` code need the whole thing restarted, since that half isn't watched by Vite.

8.2 Glossary — quick answers if your teacher asks "what is...?"

TERM	ONE-LINE DEFINITION
Electron	Packages Chromium + Node.js so a webpage can run as a desktop app with OS-level abilities.
Main process	The Node.js half of an Electron app — full OS access, exactly one per app.
Renderer process	The sandboxed Chromium tab showing the actual React UI.
IPC	"Inter-Process Communication" — the message-passing system the renderer uses to ask the main process to do something on its behalf.
Preload script / contextBridge	The narrow, explicit allow-list of functions the renderer is permitted to call into the main process.
React component	A function that returns JSX (UI) — reusable, like a custom HTML tag.
JSX	HTML-looking syntax written directly inside JavaScript/TypeScript files.
Props	Data passed into a component, like attributes on a tag.
State / hook	A value that, when changed, tells React to re-render; <code>useState</code> / <code>useEffect</code> are the built-in hooks that manage this.
Zustand store	A small shared object any component can read/write, with automatic re-rendering when it changes.
Vite	The local dev server + build tool — hot reload while developing, one optimized bundle when shipping.
Tailwind CSS	Pre-made utility CSS classes composed directly in markup, instead of hand-written custom classes.
TypeScript	JavaScript plus compile-time type checking; compiles down to plain JavaScript.
npm workspace	This repo's mechanism for treating <code>renderer/</code> as its own sub-project with its own <code>package.json</code> , while still being installed/run from the root.

Prepared as a study/presentation aid for FluencyOS's frontend, Electron shell, and onboarding flow. Every code excerpt above is taken directly from the project (lightly trimmed for length) — file paths are given so each one can be opened and shown live during a demo.